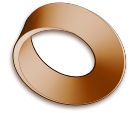


Name: _____

Date: _____

Mr. Rawson • Y9 MYP Design



3a.4 – Shapes and Parameters

Unit 3a: Interactive Animations & Games

Learning intentions

By the end of this lesson you can:

- say what a **parameter** is, and why a block needs more than one;
- draw rectangles and ellipses of **any size** using **w** and **h**;
- explain why `rect` and `ellipse` measure their size differently;
- use `background()` without wiping out your own picture.

1 What is a parameter?

Last lesson you told a shape *where* to go. Two numbers were enough: `rect(200, 150)`. But every rectangle came out the same size, because you never had a way to say how big.

A **parameter** is a piece of information you hand to a block to change how it does its job. Each opening in a block is one parameter. More openings means more control.

Parameter — additional information provided as input to a block, to customise what it does.

`rect(x, y)` → `rect(x, y, w, h)`

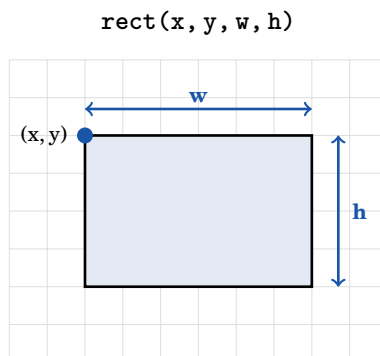
Two new parameters: **w** for width and **h** for height. Both are measured in pixels, the same units as the grid.

The order of the numbers is the meaning

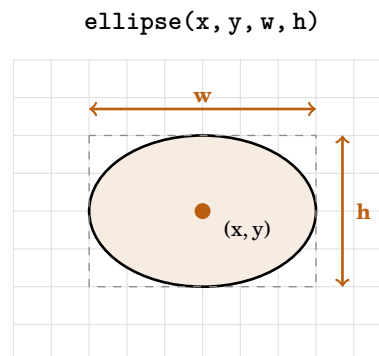
The computer does not read the names **x**, **y**, **w**, **h**. It only counts positions: first number, second number, third, fourth. Swap two of them and you get a different shape — and **no error message**, because you have not broken any rule. You have simply asked for something else.

2 The trap: `rect` and `ellipse` do not measure the same way

Both blocks take four numbers that look identical. They do not mean the same thing, and this is the thing that goes wrong this lesson.



starts at the **top-left corner**
and grows **right and down**



sits on its **centre**, and **w** and **h**
measure **all the way across**

Work one out. `ellipse(200, 200, 100, 100)` draws a circle 100 pixels wide, centred on 200. So it reaches from **150 to 250** — 50 pixels either side of the centre, not 100 pixels to the right.

`rect(200, 200, 100, 100)` reaches from **200 to 300**, because the corner is the starting line.

Same four numbers. The two shapes do not overlap.

3 `background()` paints over everything

`background(color)` fills the *entire* canvas with one colour. It takes one parameter — the colour — and it obeys the same rule you met last lesson: later instructions cover earlier ones.

Because it covers the whole 400 by 400 window, it covers *everything* you have already drawn.

Put `background()` on the first line. If your picture runs perfectly and then the screen goes blank, you have not lost your code — you have painted over it. Scroll to the bottom and look for the background block.

4 Check yourself

1. In `rect(50, 60, 70, 80)`, which number is the height? *80 — the fourth number.*

2. A square 40 pixels wide, with its corner at (100, 100). Write the code.

`rect(100, 100, 40, 40)` — a square, so width and height are the same. It reaches from 100 to 140 in both directions.

3. `ellipse(300, 100, 80, 80)` — how far does the circle reach to the *left* of its centre?

40 pixels, out to $x = 260$. The 80 is the width all the way across, so only half of it lies on each side of the centre.

4. You draw a shape, then a background, then another shape. How many shapes can you see, and why?

One — the last one. The background is drawn after the first shape, so it covers it completely. Only the shape drawn after the background survives.

5. Why does swapping two parameters give you a wrong picture instead of an error message?

Because the computer reads the numbers by position, not by name. Four numbers in four slots is a perfectly legal instruction — it just describes a different shape from the one you meant. The code is not broken, so there is nothing for it to complain about.

Activity

Now go to Code Studio — Unit 3a, Lesson 4. Work through the levels in order. **Level 1 is a prediction** — write your guess down *before* you run the code, then see how close you were. **Level 8 is the one that is marked.** Level 9 is the challenge, if you get there.

Bugs are normal and everybody gets them. Read the numbers in your block out loud in order — “x, y, width, height” — and most of them find themselves.